

Laporan Praktikum 6 Kontrol Cerdas

Nama : Teuku Muhammad Rafli

NIM : 224308022

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/rafli969>

Student Lab Assistant : RizkyPutri Ramadhani (214308092)

1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

2. Tujuan Percobaan

Tujuan dari percobaan praktikum Canny Edge Detection & Lane Detection with Instance Segmentation yaitu:

1. Memahami konsep Canny Edge Detection sebagai metode dasar deteksi
2. Menggabungkan metode Canny Edge Detection dengan Instance Segmentation untuk meningkatkan deteksi jalur.
3. Menggunakan Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection).

3. Landasan Teori

OpenCV (Open Source Computer Vision Library) adalah pustaka open source yang dikembangkan oleh Intel dan digunakan untuk pemrosesan citra digital secara real-time, serta dapat dimanfaatkan secara bebas oleh programmer Python maupun C. Salah satu teknik populer dalam pengolahan citra adalah Canny Edge Detection, yaitu algoritma deteksi tepi yang dikembangkan oleh John F. Canny, yang bekerja melalui tiga tahap utama untuk menghasilkan tepi yang bersih dengan meminimalkan noise melalui proses seperti pemburaman gambar. Python sendiri merupakan bahasa pemrograman yang serbaguna dan banyak digunakan dalam berbagai bidang, termasuk pengembangan web, analisis data besar, dan komputasi ilmiah, dengan kelebihan berupa sintaks yang sederhana dan dukungan lintas platform seperti Windows, Mac, Linux, hingga Raspberry Pi. Dalam mendukung pengolahan data numerik, pustaka NumPy menawarkan struktur array multidimensi yang lebih efisien dan cepat dibandingkan list biasa pada Python. Dalam aplikasi computer vision seperti sistem bantuan pengemudi (ADAS) dan kendaraan otonom, teknologi lane detection digunakan untuk mengenali marka jalan dalam citra atau video, dan instance segmentation berperan penting dalam membedakan setiap jalur secara individu dengan presisi piksel. Model deep learning seperti Mask

R-CNN dan DeepLab sering dimanfaatkan dalam proses segmentasi dan klasifikasi tersebut untuk mendapatkan hasil yang lebih akurat dan detail.

4. Analisis dan Diskusi

- Analisis

1. **Instance Segmentation** umumnya memberikan performa yang lebih unggul dibandingkan dengan **Canny Edge Detection** dalam skenario yang rumit. Hal ini disebabkan oleh perbedaan pendekatan antara keduanya. Canny Edge Detection cenderung kurang efektif dalam kondisi kompleks karena sensitif terhadap noise, tidak mampu memahami konteks visual secara mendalam, dan kesulitan dalam menyesuaikan diri terhadap variasi lingkungan. Sebaliknya, Instance Segmentation memiliki kelebihan dalam mengenali konteks citra, lebih tahan terhadap noise serta perubahan pencahayaan, mampu mendeteksi berbagai objek secara simultan, dan lebih handal dalam menghadapi situasi yang menantang secara visual.
2. Mengombinasikan Instance Segmentation dengan Canny Edge Detection dapat menjadi strategi yang efektif untuk meningkatkan akurasi deteksi jalur, terutama dalam menciptakan sistem yang lebih stabil dan efisien. Pendekatan gabungan ini memungkinkan kedua teknik saling melengkapi: Instance Segmentation menjadi komponen utama untuk memahami konteks secara keseluruhan, sementara Canny Edge Detection berperan dalam mempercepat proses dan menangkap detail halus, terutama dengan bantuan pendekatan berbasis *Region of Interest* (ROI). Meski begitu, keberhasilan integrasi sangat bergantung pada implementasinya. Dalam kasus yang sangat kompleks seperti area persimpangan, Instance Segmentation tetap menjadi metode dominan, sedangkan Canny berfungsi sebagai pelengkap. Untuk memastikan sistem dapat bekerja secara optimal, diperlukan eksperimen menyeluruh, penyesuaian parameter yang tepat, serta pengujian di berbagai kondisi jalan guna meningkatkan adaptabilitas terhadap lingkungan nyata.
3. Dalam algoritma Canny Edge Detection, pengaturan parameter sangat memengaruhi kualitas hasil deteksi tepi. Dua parameter utama yang perlu diperhatikan adalah **threshold bawah (T1)** dan **threshold atas (T2)**, yang masing-masing mengatur sensitivitas terhadap perubahan intensitas tepi. Threshold bawah digunakan untuk menyaring tepi dengan intensitas rendah—piksel di bawah nilai ini akan diabaikan. Sementara threshold atas menangkap tepi yang lebih kuat, di mana piksel dengan intensitas di atas nilai ini langsung dikategorikan sebagai tepi yang jelas. Piksel yang memiliki intensitas di antara kedua nilai tersebut akan tetap dianggap sebagai bagian dari tepi jika terhubung dengan tepi yang kuat, melalui proses yang dikenal sebagai *hysteresis thresholding*.

- Diskusi

1. Pemilihan antara **Canny Edge Detection** dan **Instance Segmentation** sangat bergantung pada kondisi visual dari gambar, tujuan dari proses deteksi, serta keterbatasan sumber daya komputasi. Apabila dibutuhkan deteksi yang cepat, ringan, dan hanya bertujuan untuk mengidentifikasi kontur atau struktur dasar objek, terutama dalam gambar dengan kontras tinggi dan kompleksitas rendah, maka Canny Edge Detection menjadi pilihan yang lebih efisien dan tepat.
2. Untuk meningkatkan ketepatan dalam deteksi jalur menggunakan **YOLOv8-seg**, penyesuaian parameter dapat dilakukan baik saat proses pelatihan maupun pada saat inferensi. Pada fase pelatihan, akurasi model dapat ditingkatkan melalui penggunaan dataset yang beragam dan berkualitas tinggi, penerapan teknik augmentasi data, serta penyetelan

hyperparameter agar model lebih stabil dan andal. Sementara itu, selama inferensi, mengatur nilai threshold untuk *confidence* dan *Intersection over Union (IoU)* dapat membantu memperoleh deteksi yang lebih presisi. Selain itu, hasil segmentasi juga dapat diperbaiki dengan menerapkan teknik *post-processing*, seperti penyaringan berdasarkan area objek atau penggabungan dengan hasil dari Canny Edge Detection untuk meningkatkan akurasi secara keseluruhan.

3. **Canny Edge Detection** dan **YOLOv8-seg** dapat diintegrasikan dalam sistem navigasi otomatis pada kereta dengan menetapkan fungsi spesifik untuk masing-masing metode. Dalam sistem ini, YOLOv8-seg digunakan untuk mendeteksi jalur utama yang harus diikuti, sedangkan Canny Edge Detection difungsikan untuk mengekstraksi garis tepi rel. Kolaborasi antara kedua metode ini memungkinkan sistem menghasilkan jalur navigasi yang akurat, sehingga kendaraan otomatis seperti kereta dapat bergerak secara efisien dan terarah sesuai lintasan yang telah ditentukan.

5. Assignment

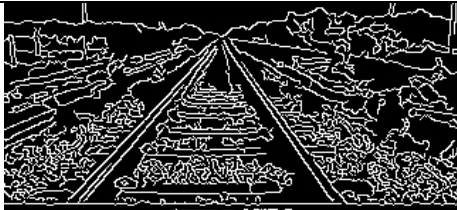
Program ini bertujuan untuk mendeteksi tepi menggunakan metode Canny Edge Detection serta mengombinasikannya dengan deteksi objek atau segmentasi menggunakan model YOLO dari Ultralytics dalam video real-time dari kamera.

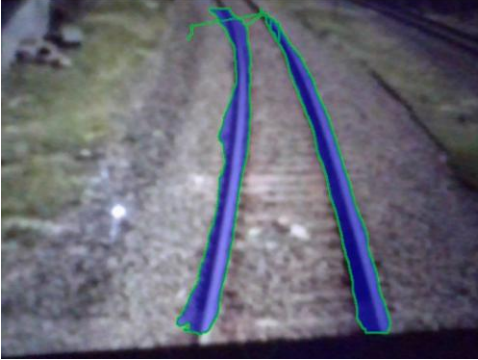
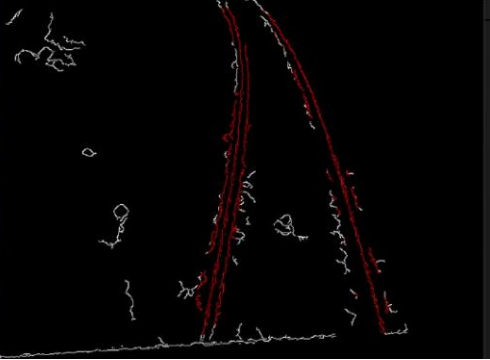
Program dimulai dengan memuat model YOLO dari path yang telah ditentukan, lalu mengakses kamera menggunakan `cv2.VideoCapture(0)`. Setiap frame dikonversi ke skala abu-abu dan diproses dengan Canny Edge Detection untuk menghasilkan gambar biner dengan tepi putih di latar belakang hitam. Secara paralel, frame asli dikonversi ke format RGB dan diproses oleh YOLO untuk mendeteksi atau melakukan segmentasi objek.

Jika YOLO menggunakan segmentasi, objek akan ditandai dengan poligon biru transparan, sedangkan jika hanya mendeteksi objek, bounding box hijau akan digambar di sekitar objek, dengan label jika tersedia. Masking digunakan untuk membedakan tepi objek yang dikenali YOLO dan tepi dari metode Canny, di mana tepi dalam area objek YOLO diwarnai merah, sementara tepi lainnya tetap putih. Hasil deteksi YOLO dan Canny Edge ditampilkan berdampingan untuk perbandingan visual.

Program berjalan hingga pengguna menekan 'q', setelah itu kamera dilepaskan dan semua jendela ditutup untuk mencegah kebocoran memori. Kombinasi deteksi tepi tradisional dan deep learning ini dapat diterapkan dalam berbagai aplikasi seperti deteksi jalur rel atau pengawasan lalu lintas.

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil Pengamatan	
1	Canny Edge Detection		

2	Lane Detection		
3	Combined		

7. Kesimpulan

Berdasarkan hasil praktikum dan analisis yang dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

1. Teknik **masking** dimanfaatkan untuk membedakan antara tepi objek yang dikenali oleh YOLO dan tepi-tepi umum yang dihasilkan oleh metode Canny Edge Detection, sehingga membantu menghasilkan visualisasi yang lebih terstruktur dan mudah dipahami.
2. Program berhasil mengintegrasikan metode Canny Edge Detection dengan model YOLO, memungkinkan proses deteksi tepi sekaligus pengenalan serta penyorotan objek secara real-time dalam video.
3. Sistem dirancang agar dapat berjalan secara berulang (loop) hingga pengguna memutuskan untuk keluar, dan telah dilengkapi dengan manajemen memori yang efisien untuk mencegah terjadinya kebocoran sumber daya.

8. Saran

Untuk meningkatkan kinerja dan tingkat akurasi sistem, program dapat dioptimalkan melalui pemanfaatan GPU, yang memungkinkan proses prediksi menggunakan YOLO berjalan lebih cepat dan responsif. Selain itu, penerapan langkah *preprocessing* tambahan seperti Gaussian Blur sebelum proses Canny Edge Detection dapat membantu mereduksi noise yang tidak relevan, sehingga hasil deteksi tepi menjadi lebih bersih dan akurat. Dari segi tampilan, penyesuaian tingkat transparansi pada lapisan warna (overlay) dapat memperjelas objek yang terdeteksi tanpa mengaburkan detail penting pada gambar. Sebagai nilai tambah, program juga dapat dilengkapi dengan fitur penyimpanan hasil deteksi dalam bentuk gambar atau video, sehingga hasil analisis dapat didokumentasikan dan dimanfaatkan lebih lanjut dalam berbagai aplikasi di dunia nyata.

9. Daftar Pustaka

Husain, B. H., Osawa, T., Astiti, S. P. C., Frianto, D., Nandika, M. R., & Agustine, D. (2024). Deteksi Lubang Jalan Secara Otomatis dari Rekaman Drone Menggunakan Model Machine Learning Berbasis YOLOv5 Instance Segmentation di Kota Pekanbaru, Provinsi Riau, Indonesia. *Jurnal Zona*, 8(2), 137–146.

Luh Putu Risma Noviana, I Putu Eka Indrawan, & Gde Iwan Setiawan. (2024). ANALYSIS OF CANNY

EDGE DETECTION METHOD FOR FACIAL RECOGNITION IN DIGITAL IMAGE PROCESSING. Jurnal Manajemen dan Teknologi Informasi, 15(2), 29–34.